LLD Notes — Phase 3, Part 5

Chain of Responsibility · Template Method · Facade · Jumbled round 3 (Jhatpat) | Stories, your doubts, your mistakes, interview answers

> **Colour key. Blue = a doubt you asked · Red = a mistake you made · Green = rule to remember · Amber = how to say it in an interview.**

Contents

21. Chain of Responsibility — the leave application · handle or pass on · two flavours · exercise 27 (Kharcha) · one class, many links

22. Template Method — three reports, same steps · the fixed skeleton · is-a vs has-a · exercise 28 (Pravesh) · ask the child a question

23. Facade — Suresh the travel agent · one front door · exercise 29 (Swasth) · five patterns in one file · cleaning up in reverse

24. Jumbled round 3 (Jhatpat) — fourteen requirements, all twelve patterns · `placeOrder()` · defending the design

25. Cheat sheet for all twelve patterns · new rules · mistake tracker · recall questions

21. Chain of Responsibility

The story: the leave application

You want leave. One day: your team lead signs it. A week: the team lead can't, so it goes up to the manager. A month: it goes up again, to the director. **You hand the form to one person.** You don't need to know who will finally sign it.

> **Your guess, before any definition: "it will be a chain, I guess."** Correct. Each person does one of two things: **handle it and stop**, or **pass it on**.

The smallest form

```
interface Approver { String approve(Claim claim); }

class LimitApprover implements Approver {
    private final String title;
    private final int limit;
    private final Approver next;                    // the person above me

    LimitApprover(String title, int limit, Approver next) {
        this.title = title; this.limit = limit; this.next = next;
    }

    public String approve(Claim claim) {
        if (claim.amount() <= this.limit) return this.title;   // I can: handle and stop
        return next.approve(claim);                            // I can't: pass it on
    }
}

class NoApproverFound implements Approver {        // the end of the chain always answers
    public String approve(Claim c) {
        throw new ClaimRejectedException("Nobody can approve " + c.amount());
    }
}
```

```
Approver chain =
    new LimitApprover("Team Lead", 5000,
        new LimitApprover("Manager", 25000,
            new LimitApprover("Director", 100000,
                new NoApproverFound())));

chain.approve(claim of 18000)
  → Team Lead:  18000 <= 5000?   no  → pass on
  → Manager:    18000 <= 25000?  yes → "Manager"      (the Director is never asked)
```

The limits above are an example. The shape is what matters: the sender holds only the first link.

Your three doubts about `LimitApprover`

> **1. "Shouldn't I create a separate class for each title?"** No. Team lead, manager and director all do the **same thing**: compare the amount with a limit. They differ only in **data** (a title and a number).
> **Objects that differ by data → one class with fields. Objects that differ by behaviour → separate classes.**

> **2. "But approve() will have if-else blocks to check each role's condition, right?"** No. Each object checks only **its own** limit: one `if`. The "else" is the next link. The if-else ladder you were picturing still exists, but it's spread across the objects, one rung each.

> **3. "I still don't understand how the class will be written" … "ooh, understood, the limit is what we are checking."** That was the click: `claim.amount() <= this.limit`. The same line runs in every link, with a different `limit` each time.

Two flavours

	"First one who can, does"	"Everyone must agree"
Each link	handles and **stops**, or passes on	checks, then passes on; any link can **reject**
Who ends up acting	exactly one link	all of them, unless one says no
Examples	approvals, support levels, an ATM picking notes	validation checks, login filters
You already use it	an exception climbing the call stack until a `catch` takes it	the Spring Security filter chain

Decorator vs Chain

Both are "an object that holds the next object of the same type", so they look alike in code.

	Decorator	Chain of Responsibility
Does every link run?	**Yes**, always, all of them	**No**, a link can stop it
What a link does	**adds** something to the result	**decides** who handles it
When the work happens	on the way **back out**	on the way **in**

The test: is each link **adding** or **deciding**? Adding → Decorator. Deciding → Chain.

Your doubt: "In Decorator the base is innermost. In Chain the first approver is outermost. Why?" Because of *when* the work happens. A decorator calls inward first and does its work on the way back, so the innermost object finishes first. A chain link does its work **before** it calls the next one, so the outermost object goes first. Same wiring, opposite timing.

Exercise 27 — Kharcha (expense claims)

Part A: 4 out of 5.

The chain must end with a link that answers. Without `NoApproverFound`, the last real approver calls `next.approve()` on `null` and the user sees a `NullPointerException` instead of "nobody can approve this."

Order is everything, and nothing checks it for you. Put the Director first and the Director approves every claim, because every amount is under the biggest limit. It compiles and runs; it's just wrong.
You suggested giving each approver a range (a lower and an upper limit). That works, but now two numbers per link must be kept in step. Simpler: **build the chain in one place** (a factory) from a list sorted by limit, so a wrong order can't be written.

Your Part B — one thing to fix. When nobody could approve, the code **returned a sentence** instead of throwing. The caller gets a String either way and can't tell "Manager" from "nobody can approve this" without reading the text.
A failure must look like a failure: throw, don't return a sentence. (You applied this correctly in the next exercise.)

Pressure points
Change: "Add a VP level between Manager and Director." One new line where the chain is built. No class changes.
Failure: "A claim bigger than every limit?" The end link throws a clear exception.
Defend it: "Three ifs would do." — "You'd be right for three fixed levels, and I'd start there. I'd move to a chain when the levels change often, differ per department, or need to be configured without a code change."

22. Template Method

The story: three reports, same steps

Three reports all do the same thing in the same order: open a connection, fetch the data, format it, send it. Only the fetching and the formatting differ. Three developers write three classes, and one of them forgets a step.

> **Your instinct: "the common steps go in an abstract class."** Right. Template Method is that instinct plus one rule: **the parent owns the order**, and the children can't change it.

The fixed skeleton

```
abstract class Admission {

    public final void admit(Student student, PaymentMethod payment) {   // final: nobody reorders this
        verifyDocuments(student);                                       // fixed step
        if (!isEligible(student))                                       // the child answers
            throw new NotEligibleException(student.name() + " is not eligible");
        payment.collect(fee());                                         // the child supplies the number
        afterFeePaid(student);                                          // hook: optional
        issueIdCard(student);                                           // fixed step
    }

    protected abstract boolean isEligible(Student student);   // required: every child must answer
    protected abstract int fee();                             // required
    protected void afterFeePaid(Student student) { }          // hook: empty by default

    private void verifyDocuments(Student s) { ... }           // private: children can't touch it
    private void issueIdCard(Student s) { ... }
}

class EngineeringAdmission extends Admission {
    protected boolean isEligible(Student s) { return s.marks() >= 75; }
    protected int fee() { return 120000; }
}
```

Kind of step	Keyword	Meaning
The skeleton	`public final`	the order is decided once, by the parent
Fixed step	`private`	the same for everyone; children can't change or even see it
Required step	`protected abstract`	every child **must** supply it, or it won't compile
Optional step (hook)	`protected`, empty body	a child **may** override it. Name it by *when* it runs: `afterFeePaid`

You already use it: `HttpServlet`. Its `service()` method is the skeleton; you only write `doGet()` and `doPost()`. Every JUnit test is the same: the framework runs set-up, your test, tear-down, in that order.

Is-a and has-a (re-taught, because the two got swapped)

> **Asked "Template Method vs Strategy: which uses inheritance and which uses composition?", you said: "I don't know what the answer for this is."** That was the honest answer, and it's why we stopped and went back to basics.

	Is-a (inheritance)	Has-a (composition)
In Java	`class Car extends Vehicle`	a **field** holding another object
Say it aloud	"a car **is a** vehicle"	"a car **has an** engine"
Can it change while running?	No, fixed when the class is written	Yes, put a different object in the field

	Template Method	Strategy
Built with	**inheritance**: the child *is an* Admission	**composition**: the service *has a* strategy
What varies	a few **steps** inside a fixed process	the **whole** calculation
Who is in charge	the parent calls the child	the caller picks the strategy
Swap at run time?	No	Yes

> **Notice:** the reference code uses **both**. The admission *process* is a template (fixed order). The *payment method* is passed in as a strategy (UPI, card, cash: the caller chooses).

Exercise 28 — Pravesh (college admissions)

Part A: 4 out of 6. Part B compiled and ran, and it threw on failure instead of returning a sentence.

> **Your doubt: "In main, where exactly are you sending the amount to be paid?"** Nowhere, and that's the point. `main` only says *how* to pay: `admission.admit(student, new UpiPayment())`. The **parent** asks the child "what is your fee?" and collects it itself: `payment.collect(fee())`. If the caller sent the amount, a caller could send the wrong one.

Your Part B: the child had `collectFee()` instead of `fee()`.

```
protected abstract void collectFee(PaymentMethod p);   // yours: the child is given a JOB
protected abstract int  fee();                         // better: the child is asked a QUESTION
```

With `collectFee`, every child repeats the same collecting code, and one of them can forget to collect, or collect twice. With `fee()`, the child only supplies a number and the parent does the collecting, once, in the right place.

Ask the child a question; don't give it a job. A child should supply only what differs. Here the **amount** differs; the **collecting** doesn't.

Pressure points

Change: "MBA admissions need an interview after the fee." If only MBA needs it: override the hook. If the step sits somewhere new: add one new empty hook in the parent. No other child changes.

Failure: "Fee collected, then the ID card step fails." The same answer as everywhere: the class that knows the steps (here the parent) refunds, then rethrows.

Defend it: "Why not Strategy?" — "You'd be right if the whole algorithm varied or had to be swapped at run time. Here the order is fixed and must not be changed by anyone; only two values differ. A final method in a parent class guarantees the order."

23. Facade

The story: Suresh the travel agent

A trip to Goa needs a train ticket, a hotel, a cab and a ferry pass. You could ring four offices yourself, in the right order, and cancel the hotel if the train is full. Or you tell Suresh "Goa, three days" and he does all of it. **Suresh has no new powers.** Every office is still open to you. He just gives you one simple door.

Your guess: "one service that the others call." Yes. And you've already written one: `BookingService.bookTicket()` in PuneGo was a facade over the factory, the decorators, the payment gateway and the listeners.

A facade…	Meaning
adds no new ability	it only calls things that already exist, in the right order
doesn't lock the door	callers who need one part can still use that part directly
protects callers from change	swap the payment provider and the desk, the app and the bot don't notice
has one danger	everything wants to land in it, and it grows into a **god class**

Three wrappers, one line each (you got all three):
Adapter: one thing, wrong shape → change the shape.
Decorator: one thing, right shape → add behaviour.
Facade: many things → one simple front door.

Exercise 29 — Swasth (hospital software)

Req	What it said	Answer	You
R1	Booking = five steps, needed by desk, app and phone bot	Facade	✓
R2	Lab machine's `fetchRes(int)` returns a comma string	Adapter	✓
R3	"Put the fee rules in the booking service, everyone calls it"	**No.** A fee class behind the facade	½
R4	"Route the pharmacy's one stock call through the facade"	**No.** Call it directly	✓
R5	BOOKED → CHECKED_IN → WITH_DOCTOR → DONE, plus two more	State	✓

R3 — right classes, wrong home (it happened twice). You saw that the discounts stack and reached for decorators, which is fine. But you built them **inside** the booking method, in Q1 and again in Q2. Next year the billing desk needs the same fee for a walk-in patient, and can't get it without booking something.

The facade test: every line in it should be a call to something else. If you see arithmetic or business rules in there, they've moved into the wrong class. The facade's whole involvement in the fee is one line:

```
int fee = feeCalculator.feeFor(patient, doctor);      // the facade asks; it doesn't calculate
```

R4 — a facade earns its place by hiding several steps. Wrapping a single call adds a layer and hides nothing. **You said this yourself.**

Q2 — your first draft of `bookAppointment()`

✓ **The text message went last, through listeners, and you threw on failure.**

You wrote	Problem
`if (slotAvailable())`	You **checked** the slot but never **reserved** it. Two patients can get the same slot.
`new WithinSevenDaysDiscount(new SeniorFeeDiscount(...))`	R3 again: fee rules built inside the facade.
`.collect()` on the fee object	Working out the fee and taking the money are two jobs.
`void`, no slot parameter, `throw NoAvailable...()`	The caller gets nothing back; which slot?; `new` is missing.

Q3 ✓ "Payment fails after the slot is reserved: free the slot." Correct, and it belongs in the facade, because only the facade knows a slot was reserved a moment ago. One line to add: **after cleaning up, rethrow.** Without it the code carries on, creates a visit record, and texts the patient about an appointment that doesn't exist.

Q4 ✓ "The facade has 40 methods." You gave both halves: move any business logic back out, and split by subject (appointments, lab, insurance). The name for what went wrong is a **god class**.

Say it: "A facade should cover one area. When it grows past that, I'd split it into one facade per area and push any logic back into the classes behind it."

Your Part B — five patterns, one file, first time

It compiled with no fixes and all 21 lines of output matched exactly. You wrote seven classes: an adapter, two decorators, the fee calculator, two states, and the facade with its listeners.

Your facade, with fix 2 below already applied:

```
class AppointmentFacade {
    private final List<BookingListener> listeners = new CopyOnWriteArrayList<>();
    private final SlotService slotService;
    private final FeeCalculator calculator;
    private final PaymentService paymentService;
    // constructor sets the three services

    public void subscribe(BookingListener listener) { listeners.add(listener); }

    public Appointment book(Patient patient, Doctor doctor, String slot) {
        if (!slotService.isFree(slot)) throw new SlotNotAvailableException(slot);   // 1 required
        slotService.reserve(slot);                                                  // 2 required
        int fee = calculator.feeFor(patient, doctor);        // Decorators live in there, not here
        try {
            paymentService.collect(patient, fee);                                   // 3 required
        } catch (PaymentFailedException e) {
            slotService.release(slot);                       // clean up
            throw e;                                         // then rethrow
        }
        Appointment appointment = new Appointment(patient, slot);                   // 4 required
        for (BookingListener listener : listeners) {                                // 5 announce last
            try { listener.onBooked(appointment); }
            catch (RuntimeException e) { System.out.println("Listener failed: " + e.getMessage()); }
        }
        return appointment;
    }
}
```

```
Slot 10:30 reserved
Collected 400 from Baba                    ← senior: 500 less 20%
SMS to Baba: appointment at 10:30
Listener failed: dashboard is down         ← one listener fails, the booking stands
Slot 10:00 is taken                        ← Dev: no slot, no SMS
Slot 11:30 reserved
Slot 11:30 released                        ← Meera: reserved, payment failed, released
Payment failed for Meera
```

Three small things (none changed the output).
1. A spec detail. `"Already Checked in"` where the spec said `"Already checked in"`. `main` never triggered that line, so the output couldn't catch it. Read your strings against the spec.
2. The catch was too wide. You wrote `catch (Exception e)` and threw a **new** `PaymentFailedException`. A bug inside `collect` (a `NullPointerException`, say) gets relabelled "payment failed" and the real cause is hidden. Catch only what you expect, and rethrow the same exception.
3. Money and `double`. `(int) (amount * 0.8)` works here, but decimals aren't exact and the cast cuts off instead of rounding. Stay in whole numbers: `amount * 80 / 100`.

Your doubt: "I need to create a record for the booking listener, right?" No. A **record holds data** (`Patient`, `Doctor`). A **listener is behaviour**, so it's an interface. And because that interface has only one method, each lambda in `main` already *is* a listener. Your part was only the list, `subscribe()`, and the loop.

Part C

Q1 ✓ "A sixth step: block the doctor's Google Calendar; if it fails the booking stands." You: a calendar listener, and `book()` doesn't change. Exactly. Two additions: Google Calendar is someone else's API, so put an **adapter** between the listener and it; and a failed block shouldn't vanish silently, so log it or retry it.

Q2 ½ — "Payment succeeds, then creating the appointment fails." You said: catch it in the appointment class, note the failure in a database, and let a scheduler retry. Retrying is a real technique, but three gaps:
1. The appointment class doesn't know a payment happened. Only the **facade** knows all the steps.
2. What does the patient see right now? Never report a partial failure as success.
3. The database is down, and you're writing the failure to a database.

```
slotService.reserve(slot);
try {
    paymentService.collect(patient, fee);
    try {
        appointment = records.create(patient, slot);
    } catch (RuntimeException e) {
        paymentService.refund(patient, fee);    // undo step 3
        throw e;
    }
} catch (RuntimeException e) {
    slotService.release(slot);                  // undo step 2
    throw e;
}
```

When something fails halfway, the class that knows all the steps cleans up, in reverse, and then rethrows. Last done, first undone, the same as undoing a macro command.

Say both: "By default I'd refund and release, in reverse order. If the business prefers, I'd keep the payment, mark the booking as *pending* and retry, with a refund if the retries fail."

Q3 ½ — "Your facade is just a service class with a fancy name." Your defence was true but had no concession, and it named the wrong alternative: the god class is what the facade *becomes* if it grows. Without a facade, the real problem is the **three callers** each repeating the five steps, and one of them forgetting the cleanup.
Script: "You'd be right that it's an ordinary service class; I wouldn't add anything special to it. The name just tells the team its job: one entry point for booking, so the desk, the app and the bot don't each repeat the five steps and the cleanup. It should only coordinate. The moment it starts holding rules of its own, it's on its way to a god class."

24. Jumbled round 3 — Jhatpat

Ten-minute grocery delivery in Pune. All twelve patterns in play, no hints, three requirements that need none. You answered after a night's gap.

Q1 — you scored 13 out of 14

Req	What it said	Answer	You
R1	Every order gets an id and the time it was placed	Nothing: fields	✓
R2	Refunds: agent to Rs 200, team lead to Rs 2,000, manager above	Chain of Responsibility	✓
R3	SDK's `doTxn(paise, currency)` vs our `pay(rupees)`	Adapter	✓
R4	Delivery fee differs by type; the type arrives as text	Strategy + Factory	✓
R5	No rider has picked up the order in 15 minutes: alert	**Nothing: a scheduled job**	✗ **unsure**
R6	Cart: add, remove, change quantity, with Undo	Command	✓
R7	PLACED → PACKED → OUT_FOR_DELIVERY → DELIVERED, with rules	State	✓
R8	Placing an order = five steps; app, website and bot need it	Facade	✓
R9	Extras: gift wrap, ice pack, eco bag, any combination	Decorator	✓
R10	Three daily reports, same steps, two steps differ	Template Method	✓
R11	On delivery: SMS, points, dashboard; one may fail	Observer	✓
R12	Promo offer: 2 required, 8 optional, a date rule, never changes	Builder	✓
R13	"Maybe next year we'll deliver medicines"	Nothing: YAGNI	✓
R14	The stock count must be the same number everywhere	One shared instance, injected	✓

Two things that had changed: on R2 you said "one class, different objects per limit" unprompted; on R14 you said "create it once and inject it", with no Bill Pugh.

R5 — the third time this one has appeared (the ticket closed after 7 days; the PuneGo ticket unscanned for 2 hours). The clue: **nothing happens.** Observer reacts when something happens. Here the trigger is that a rider *didn't* come, so there is no event to listen to. Only a clock can notice that.

"X hasn't happened in N minutes" is a scheduled job, not a pattern. Every minute: "which orders are still PLACED and older than 15 minutes?"

R6 — right pattern, wrong shape. You described a Command interface with add item, remove item and undo. The interface has two methods: `execute()` and `undo()`. `AddItem`, `RemoveItem` and `ChangeQuantity` are three **classes** that implement it. The `Cart` is the receiver.
R1: a record can't change after it's created, and an order's status changes, so `Order` is a normal class.

Q2 — weaving `placeOrder()`

✓ **The order of the steps was right, and you left the SMS and points out** (R11 fires on *delivery*, not on placing the order).

You wrote	Problem
the factory, gateway and decorator as **method parameters**	The app, website and bot would each have to find those objects. Tools go in the **constructor**; the method takes only what the customer chose.
`return NoAvailableStockException(item)`	An exception is thrown: `throw new ...`
reserve inside the loop	Items 1 and 2 reserved, item 3 missing: you throw, and 1 and 2 stay reserved forever. Check everything, then reserve.
`deliveryFareFactory(Extras).calculateFee`	Two things merged. The delivery **type** picks a formula (Factory → Strategy). The **extras** add cost to the order (Decorator).

```
// tools come in through the constructor: stock, pricing, fees (factory), gateway, riders

Order placeOrder(Cart cart, String deliveryType, List<String> extras) {
    stock.checkAll(cart.items());                              // 1 required, throws if any is missing
    stock.reserveAll(cart.items());                            // 2 required
    try {
        int itemsCost = pricing.costOf(cart, extras);          //   Decorator lives in here (R9)
        int delivery  = fees.forType(deliveryType).fee(cart);  //   Factory → Strategy (R4)
        gateway.pay(itemsCost + delivery);                     // 3 required, Adapter behind it (R3)
    } catch (RuntimeException e) {
        stock.releaseAll(cart.items());                        //   clean up, then rethrow
        throw e;
    }
    Order order = new Order(nextId(), LocalDateTime.now());    // 4 required
    riders.assign(order);                                      // 5 a follow-up (see Q3)
    return order;
}
```

Q3 — payment succeeds, then no rider is available (½)

You said: revert everything. That's safe and consistent, and if you choose it, do it in reverse and rethrow. But ask the test question first: **if this step fails, is the order still valid?**

Step fails	Is the order still valid?	So
Payment	No. No money, no order.	required: clean up and fail
Assign a rider	**Usually yes.** One will be free in a minute.	a follow-up: keep the order PLACED and retry

The safety net already exists: it's **R5**. After 15 minutes with no rider, the scheduled job alerts the manager, and the order can be cancelled and refunded then.

> **On notifying with Observer:** if you roll back inside `placeOrder()`, the customer is still on the screen and the thrown exception *is* the notification. A listener is for the later case: the 15-minute job cancels the order after the customer has closed the app, so an "order cancelled" event with an SMS listener is right.

> **Say it as a choice:** "I'd ask the business. My default is to keep the order and retry the rider, with a time limit after which we cancel and refund."

Q4 — "Fourteen requirements, twelve patterns. Over-engineered?" (½)

Your answer (a god class is hard to extend) is true, but it would fit any design, and the concession was missing.

> **Three moves**
> **1. Concede first.** Three refund levels could be three ifs. Three extras could be three booleans.
> **2. Point at the requirements, not the future.** The Adapter is there because we can't edit the SDK. The Command is there because there's an Undo button.
> **3. Show where you said no.** Three requirements got no pattern: plain fields, a scheduled job, and YAGNI.
> **Script:** "You'd be right if I'd added them for their own sake. But each one maps to a requirement: Undo needs Command, a vendor SDK we can't edit needs an Adapter. Where nothing was needed I used nothing: plain fields, a scheduled job, and I skipped the 'maybe next year' feature. For the smaller ones, like three refund levels, I'd happily start with plain ifs and move to the pattern when the rules grow."

25. Consolidation

Requirement → pattern cheat sheet (all twelve)

What the requirement sounds like	Answer
"a different formula per type", "we change our minds twice a year"	**Strategy**
"the type arrives as text and we must pick the right object"	**Factory**
"must be the same number everywhere", "one source of truth"	**one shared instance**, injected
"2 required, many optional, a rule across fields, never changes after"	**Builder**
"when X happens also do A, B, C" and "X still counts if one fails"	**Observer**
"allowed in some statuses, refused in others"	**State**
"several extras can apply together, in any combination"	**Decorator**
"their SDK doesn't match our interface, and we can't edit it"	**Adapter**
"undo", "do it later", "replay", "retry"	**Command**
"up to X this person, up to Y that person, above that someone else"	**Chain of Responsibility**
"same steps in the same order; only one or two steps differ"	**Template Method**
"several steps in order, and three different callers need it"	**Facade**
"no A, no X" (payment before the order)	**nothing**: a required step, called directly
"X hasn't happened in N minutes"	**nothing**: a scheduled job
"every one gets an id and a time", "always the same few fields"	**nothing**: fields, or a record
"one call to one class"	**nothing**: call it directly, no facade
"we wrote it ourselves, it just needs a better name"	**nothing**: rename it
"maybe next year"	**nothing**: YAGNI

Look-alikes

Pair	How to tell them apart
Decorator vs Chain	Adding (every link runs) or deciding (a link can stop it)?
Template Method vs Strategy	A few steps vary inside a fixed order (inheritance), or the whole calculation varies (composition)?
Adapter vs Decorator vs Facade	One thing, change shape · one thing, add behaviour · many things, one front door
Observer vs required step	If it fails, is the operation still valid?
Observer vs scheduled job	Did something happen, or did something *not* happen?

New rules from this group

Area	Rule
Chain	Each link handles and stops, or passes on. The sender knows only the first link.
Chain	The chain must end with a link that answers (throws a clear exception).
Chain	Order is everything. Build the chain in one place, sorted by limit.
Chain	Differ by data → one class with fields. Differ by behaviour → separate classes.
Template	The parent's `final` method owns the order. Children can't change it.
Template	Fixed steps `private`, required steps `abstract`, optional steps are empty hooks.
Template	Ask the child a question; don't give it a job (`fee()`, not `collectFee()`).
Template	Inheritance is is-a (`extends`). Composition is has-a (a field).
Facade	Every line in a facade is a call to something else. No arithmetic, no rules.
Facade	It adds no new ability and doesn't lock the door.
Facade	One facade per area. Wrapping a single call hides nothing.
Facade	Tools come in through the constructor; the method takes only what the user chose.
Failure	The class that knows all the steps cleans up, in reverse, then rethrows.
Failure	A failure must look like a failure: throw, don't return a sentence.
Failure	Catch only what you expect, and rethrow the same exception.
Failure	Check everything first, then reserve. Don't reserve inside the checking loop.
Failure	If a later step can simply be retried, keep the order and retry, with a time limit.
Java	Money stays in whole numbers (`* 80 / 100`), never `double`.
Java	A record holds data and can't change. Behaviour is an interface; changing state is a class.
Interview	On "over-engineered": concede, point at the requirements, show where you used nothing.

Mistake tracker

Mistake	Status after this group
Missing `public`, receivers, return types, imports	**gone; Swasth compiled first time**
Recording before executing; discarding the inner result	**not seen in this group**
Reaching for Bill Pugh	**gone: "create once and inject" on Jhatpat R14**
Skipped requirement / small spec details	**smaller: one string ("Already Checked in"); the reserve step missing in a draft**
Returning a failure instead of throwing it	**Kharcha Part B; `return ...Exception` again in Jhatpat Q2**
Logic placed inside the coordinator	**Swasth R3, twice; fixed in Part B**
Catching too widely, or catching and carrying on	Swasth Part B (`catch (Exception e)`); rethrow needed a reminder in Q3
Giving the child a job (`collectFee`)	Pravesh; understood after one explanation
Observer where a scheduled job is needed	**third appearance (Jhatpat R5)**
Not conceding on "defend it"	**slipped back: Swasth Q3 and Jhatpat Q4**
`double` for money	new: Swasth and Jhatpat drafts

Where you stand. **Recognition:** Kharcha 4/5 · Pravesh 4/6 · Swasth 4½/5 · Jhatpat **13/14** across all twelve patterns, after a night's gap.
Code: five patterns in one file, compiled first time, output exact.
Two things to practise: on every exception line, check it says `throw new`. On every "defend it" question, open with **"You'd be right if…"**

Recall questions — cover the answers

1. A chain link does one of two things. Which two?
2. Team lead, manager, director: one class or three? What's the rule?
3. Write the two lines inside `LimitApprover.approve()`.
4. What goes at the end of every chain, and what happens without it?
5. The Director is wired first. What goes wrong, and does the compiler notice?
6. Name the two flavours of chain and one example of each.
7. Decorator vs Chain: what one question tells them apart?
8. Why is the base innermost in a Decorator but the first approver outermost in a Chain?
9. "Three ifs would do." Give the answer, starting with the concession.
10. In Template Method, which keyword goes on the skeleton method, and why?
11. Fixed step, required step, optional step: which keywords?
12. What is a hook, and how should it be named?
13. `collectFee()` or `fee()`? Why?
14. In Pravesh, where does `main` send the amount to be paid?
15. Is-a and has-a: which is `extends`, which is a field?
16. Template Method vs Strategy: give three differences.
17. How does the Pravesh code use a template and a strategy together?
18. Where have you already used Template Method in Java?
19. Suresh the travel agent: what new ability does he add?
20. Adapter, Decorator, Facade: one line each.
21. What is the one-sentence test for a healthy facade?
22. Why must the fee rules not live inside `bookAppointment()`?
23. The pharmacy needs one stock call. Through the facade or direct? Why?
24. A facade has 40 methods. Name the problem and give two fixes.
25. Write the six steps of `book()` in Swasth, in order.
26. Payment fails after the slot is reserved. Which two lines go in the catch?
27. Why `catch (PaymentFailedException e)` and not `catch (Exception e)`?
28. Payment succeeded, creating the appointment failed. What is undone, and in what order?
29. When is "keep the payment and retry" the better choice, and what must the patient be told?
30. Why `amount * 80 / 100` rather than `amount * 0.8`?
31. Record or interface for a listener? Why?
32. "No rider has picked up the order in 15 minutes." Why isn't that Observer?
33. The Command interface for a cart with Undo: which methods, and which classes implement it?
34. Why do a facade's tools go in the constructor and not in the method's parameters?
35. Three items in the cart, the third is out of stock. What bug does reserving inside the loop cause?
36. Delivery type and extras: which pattern handles each?
37. Paid, but no rider is free. What question do you ask first, and what's the default answer?
38. "Twelve patterns: over-engineered?" Give the three moves.
39. Which three Jhatpat requirements needed no pattern?
40. Name all twelve patterns, with one trigger phrase each.